

Column Comments Mission

Automated, standards-compliant column descriptions
for Data Lake tables

Audience: Data Engineers

Length: 20 min + Q&A

The problem

Every Data Lake table needs column-level descriptions. Today:

- 🐢 **Slow** — writing 50–200 comments per table, by hand
- 🎲 **Inconsistent** — `GCR` becomes "Gross Cash Receipts", or just `gcr`
- 📄 **Stale** — once written, comments rarely keep up with schema changes or business-term updates
- 🕵️ **Knowledge silos** — context lives in Confluence pages, Slack threads, and the head of one engineer
- 📏 **Hard limits get hit** — Hive's 256-byte COMMENT cap silently truncates anything past 255 chars

Result: column comments are either missing, wrong, or untrustworthy.

What this mission produces

A 3-stage Moon Units pipeline that:

1. **Researches** the table — DDL, Confluence design pages, Alation catalog, Certified Data Dictionary
2. **Enriches** the DDL with COMMENT clauses on every column, applying the GoDaddy Column Description Standard
3. **Validates** the 255-char limit and condenses overflowing comments

Output: a **production-ready** `table.ddl` you can PR straight into `gdcorp-dna/lake`, plus a complete audit trail.

Status today: 18 tables enriched across 9 schemas (`enterprise`, `customer360`, `ecomm360`, `analytic`, `gd_traffic_mart`, ...)

Before / After — enterprise.fact_bill_line

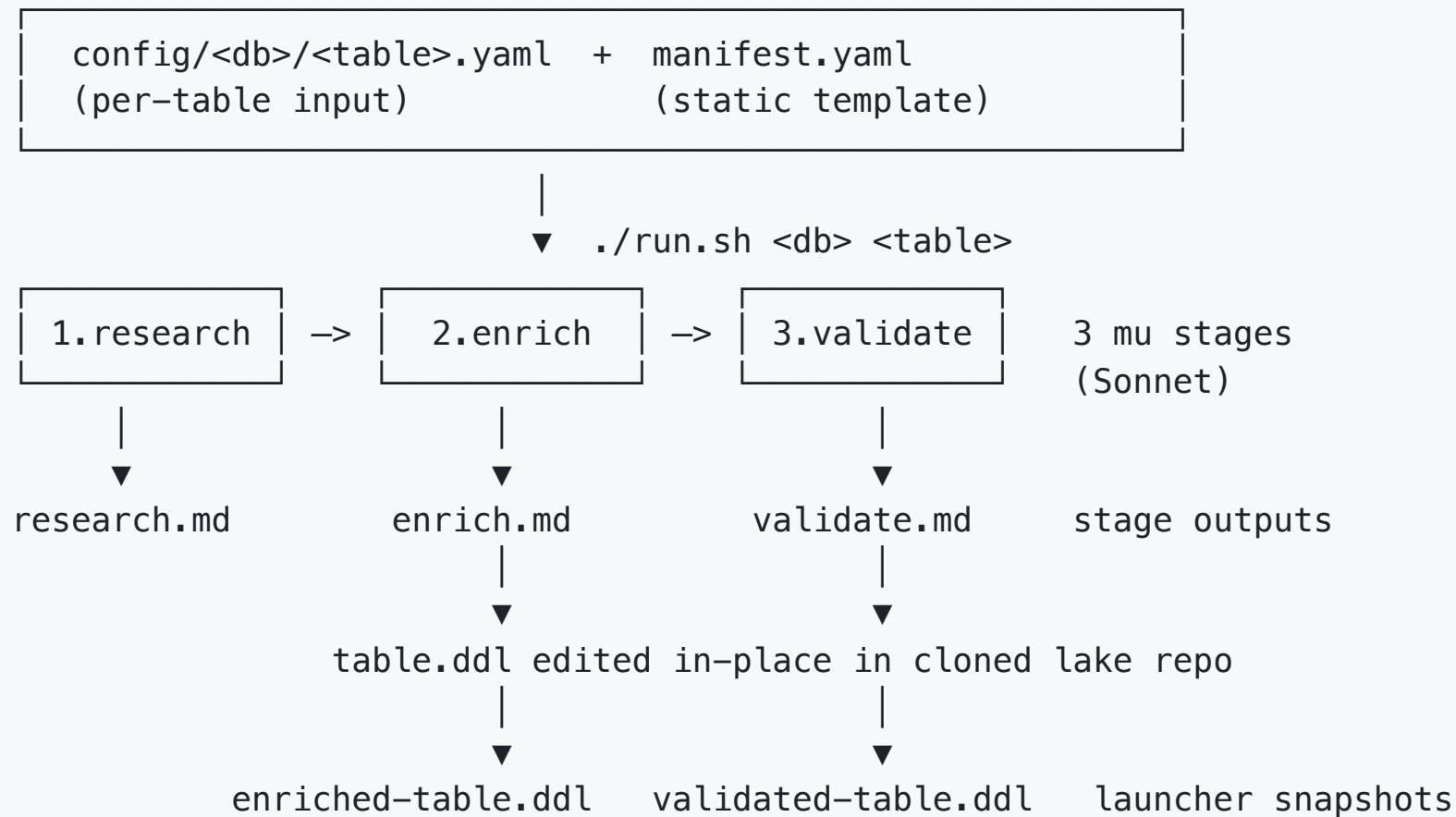
```
-- BEFORE (what's in gdcorp-dna/lake today, for many tables)
CREATE TABLE fact_bill_line(
  bill_id          string
  ,bill_line_num    int
  ,source_system_name string
  ,refund_flag      boolean
  ,gcr_usd_amt       decimal(18,2)
  -- 69 more columns, no comments
);
```

Before / After — `enterprise.fact_bill_line`

```
-- AFTER (what the mission produced – 74 columns, all annotated)
CREATE TABLE fact_bill_line(
  bill_id          string  COMMENT '@PrimaryKey Unique identifier of the
    billing receipt (order). Maps to order_id in legacy e-commerce source
    tables or subscription_order_id for Smartline. Part of composite primary
    key (bill_id, bill_line_num, source_system_name).'
  ,bill_line_num    int     COMMENT '@PrimaryKey Line item number within a
    billing receipt. Together with bill_id and source_system_name forms the
    composite primary key.'
  ,source_system_name string COMMENT '@PrimaryKey Name of the source
    e-commerce system. @Enumerated(legacy e-comm, new e-comm, Smartline
    subscription store name).'
  ,refund_flag      boolean COMMENT 'Indicates whether this bill line is a
    refund transaction (true/false). True when bill_id contains the
    character R.'
  ,gcr_usd_amt      decimal(18,2) COMMENT 'Gross Cash Receipts (GCR) for
    this line, in US dollars. Sum of receipt_price minus refunds and
    chargebacks.'
);
```

Note: `GCR` expanded from the **Certified Data Dictionary** — never paraphrased.

The pipeline at a glance



Three sources of truth feed research :

Stage 1: Research

Reads:

- The target `table.ddl` and `table.yaml` from `gdcorp-dna/lake` (cloned at start)
- All Confluence URLs listed in the per-table config
- Alation: target table metadata + columns (incl. existing `column_comment` Source Comments)
- Alation: reference tables' columns (predecessor / sibling tables)
- Certified Data Dictionary documents (paginated)

Produces `research.md` with:

- The full current DDL
- Confluence page summaries
- A **Certified Data Dictionary Mappings** table (mandatory)
- Per-column inferred purpose + context from every source

Stage 2: Enrich — applies the Column Description Standard

12 mandatory rules. The agent enforces them; we don't.

#	Rule	What it looks like
1	Be clear and concise	"Indicates whether..." not "This is the column that indicates whether..."
4	Avoid abbreviations	GCR → "Gross Cash Receipts (GCR)"
5	Indicate units & scale	...in US dollars , ...in milliseconds , ...(true/false)
8	Key annotations	@PrimaryKey , @ForeignKey(dest_table) , @Enumerated(v1, v2, ...)
10	At least one PK column	Every table must declare its primary key
11	Audit columns include TZ	etl_build_mst_ts documents Mountain Standard Time
12	Preserve 'Employee PII'	Never overwritten — appended to

Stage 2: Enrich — what the agent appends to `enrich.md`

Enrichment summary

- Target: `enterprise.fact_bill_line`
- Columns touched: 74
- Columns with pre-existing comments preserved: 3 ('Employee PII' annotations)
- Columns newly annotated: 71

Certified Data Dictionary terms applied

Abbreviation	Official expansion	Where used
GCR	Gross Cash Receipts	<code>gcr_usd_amt</code> , <code>margin_gcr_usd_amt</code>
MSRP	Manufacturer's Suggested Retail Price	<code>original_list_price_usd_amt</code>

Notable decisions

Stage 3: Validate

The 255-char enforcement layer. Every `COMMENT '...'` is recounted.

If a comment exceeds 255 chars, it's condensed using rules **in order**:

1. Drop parenthetical synonyms / aliases
2. Drop verbose qualifiers ("that is used for" → remove)
3. Shorten `@Enumerated` lists (keep top 2-3 values, add "etc.")
4. Drop secondary context sentences
5. Tighten phrasing ("Indicates whether" → "Whether")
6. Last resort: drop least-essential clause

Never mid-word truncation. **Never** `...`-ellipsis cutoff.

Output: `validate.md` reports condensed columns with before/after lengths.

How a data engineer uses it

Once per table:

```
cd missions/column-comments

# 1. Author the per-table config (use the helper scripts!)
#   config/enterprise/fact-bill-line.yaml – fills in:
#     – target.db_name / table_name / registry_path
#     – confluence_pages: [...]
#     – reference_tables: [...]
#     – alation: {enabled: true}

# 2. Run it
./run.sh enterprise fact-bill-line

# 3. Review output/enterprise/fact-bill-line/
#     validated-table.ddl      ← PR this into gdcorp-dna/lake
#     ddl-comparison.md       ← side-by-side diff
#     research.md / enrich.md / validate.md ← audit trail
```

That's it. ~5–15 minutes per table, depending on Confluence/Alation rate limits.

Authoring configs is automated too

Don't hand-write configs. Use the `column-comments-config` Claude Code skill, which chains four helper scripts:

Script	Purpose
<code>alation_fetch_table_metadata.py</code>	Pull description + custom_fields from Alation; extract Confluence URLs
<code>fetch_alation_catalog_set_design.py</code>	Read Catalog Set's shared "Data Design" link from <code>/api/v1/table/<id>/shared_catalog_sets[].description</code>
<code>confluence_search_bi_space.py</code>	CQL search of the BI Confluence space, with noise filter and excerpts
<code>lake_lineage_fetch.py</code>	Pull <code>table.yaml</code> lineage; resolve upstream deps to Alation IDs

```
# In Claude Code:  
> create a config for enterprise.dim_subscription  
# (skill chains the scripts, populates the YAML, updates CATALOG.md)
```

Full playbook: `missions/column-comments/docs/creating-a-new-config.md`

Per-table config — what humans actually edit

```
# config/enterprise/fact-bill-line.yaml
target:
  db_name: "enterprise"
  table_name: "fact-bill-line"
  registry_path: "standard" # or "dlms-api"

confluence_pages:
- url: "https://godaddy-corp.atlassian.net/wiki/spaces/BI/pages/10371978/Fact_Bill_Line"
  description: "Fact_Bill_Line design specification"

reference_tables:
- name: "fact_bill_line_vw"
  schema: "ecomm360"
  alation_table_id: 7027689
  description: "Successor table – most column descriptions are relevant"

alation:
  enabled: true
  search_query: null # defaults to db_name.table_name
```

15–40 lines per table. The whole knowledge bundle a human would assemble — encoded once.

Audit trail per run

```
output/<db>/<table>/  
├─ INPUT.md           ← parameters the mission ran with  
├─ research.md        ← stage 1: Confluence + Alation + Dictionary  
├─ enrich.md          ← stage 2: enrichment summary + decisions  
├─ validate.md        ← stage 3: 255-char check + rewrites  
├─ original-table.ddl ← snapshot pre-enrich  
├─ enriched-table.ddl ← snapshot post-enrich  
├─ validated-table.ddl ← snapshot post-validate (authoritative)  
└─ ddl-comparison.md  ← per-column Original | Enriched | Validated | Len
```

All committed to the missions repo as a permanent record.

Try it

```
git clone https://github.com/jxhuang-godaddy/moon-unit-missions
cd moon-unit-missions/missions/column-comments
cp .env.local.example .env.local          # fill in tokens
./run.sh                                  # lists configurable tables
./run.sh enterprise fact-bill-line        # actual run
```

Repo paths to read:

- `README.md` — usage
- `docs/data-flow.md` — mermaid diagrams of every stage
- `docs/creating-a-new-config.md` — config-authoring playbook
- `CLAUDE.md` — gotchas & API quirks (the war stories)

Built with: Moon Units, Claude Sonnet 4.6, Alation API, Confluence API

Questions?